Group Independent Study

Collaborative AI Agents *for* Data Mapping & Demand Forecasting

Presented by **Arial Huang, Arturo Medina, Katherine Gong, Ayda Elzohbi**

Overview

Project Context

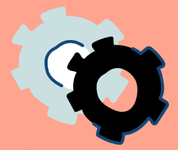
Goal:
Design and implement a two-part AI system to automate data preprocessing and forecasting model selection.

Key Challenges:
- Manual data selection, cleaning and integration across multiple sources
- Repetitive model testing to find the best forecast

Our Approach

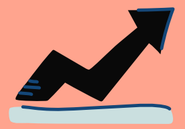
Part 1 – Data Mapping AI Agent
- Automatically profiles and maps datasets

Part 2 – Multi-Agent Forecasting System
- AI agents train, evaluate, and refine multiple forecasting models
- Feedback loop improves performance automatically


Reduced manual work


Improved forecasting accuracy


Faster insights


Scalable & adaptive workflows

Hypothesis

Core Assumptions	
Data Mapping	An AI agent can automatically map diverse data sources to target schemas with minimal manual intervention, reducing preprocessing time.
Forecasting	A multi-agent system can autonomously identify optimal forecasting models, outperforming single-model baselines.
Tight coupling between preprocessing and forecasting will improve overall pipeline adaptability.	

Expected Behaviors	
Data Mapping	Agent will map the input data to target schema and perform evaluation (and integrate evaluation outcome to the mapping process).
Forecasting	Agent will converge on best model through feedback iteration
System will generalize to new data sources without need for retraining.	

Data

Input Data

Dataset composition:

- **Business data:** Synthesized from Kaggle dataset, split into multiple sources (transactions, products, stores, promotions)
- **External factors:** Real-world data for weather, GDP, employment rates, holidays

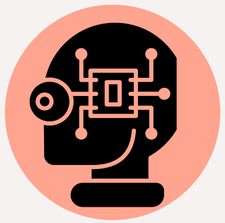
Source formats: CSV files with varying structures

File Name	Description
transaction_like_synth.csv	Daily sales records
product_like_synth.csv	Product catalog
store_like_synth.csv	Store information
promotion_like_synth.csv	Promotional campaigns
holidays.csv	Holiday calendar
GDP_monthly.csv	Economic indicators
GDP_province_yearly.csv	Economic indicators
CPI_monthly.csv	Inflation data
Population.csv	Demographics
weather_monthly.csv	Climate data
employment_data.csv	Labor market stats

Target Schema

Target: Single unified schema with 30+ fields
Table Structure: One row per date-product-store combination with all related information included.

Identifiers	Sales Performance	Product Details	Store Context	Promotions	External Factors
Date	Units Sold	Category	Store Cluster	Promotion Active	Holidays
Product ID	Unit Net Price	Subcategory	Store Size	Promotional Price	Weather
Store ID		Is Seasonal	Region		CPI
					GDP
					Unemployment

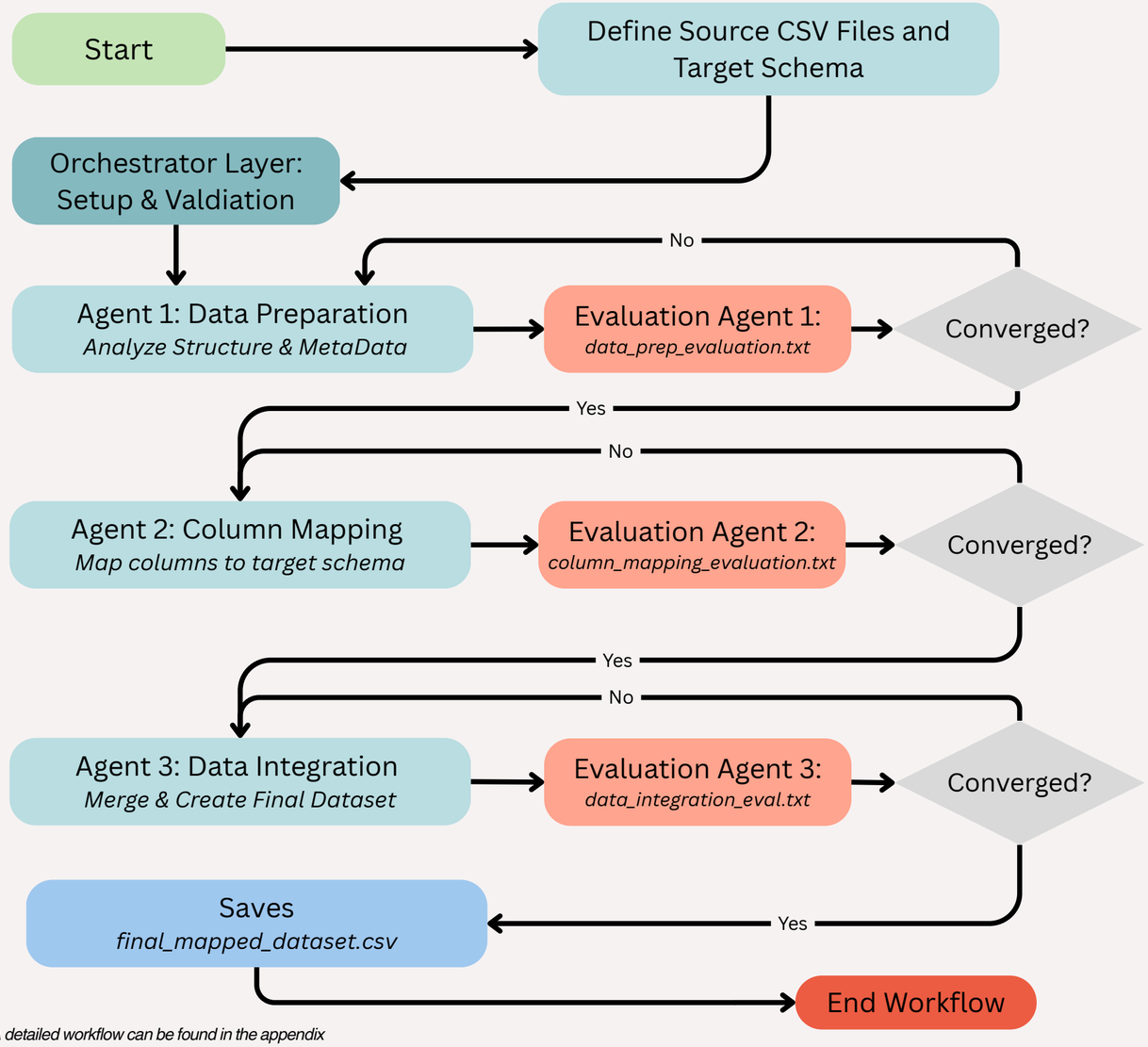


Example mappings the agent should perform:

- transaction_date → date
- product_code → product_id
- store_number → store_id

Model Methodology

Data Mapping Agent



*A detailed workflow can be found in the appendix

Overview

Automates mapping of diverse CSV data sources into a target schema.

Three specialist agents (DataPrep → ColumnMapping → DataIntegration) are interleaved with evaluator agents, all routed by the Workflow Orchestrator

Three-agent Workflow:

- **Agent 1:** Inspects every CSV, extracting schema metadata and profiling samples
- **Agent 2:** Maps source columns to target schema using semantic matching
- **Agent 3:** Merges data into compliant final dataset

Comprehensive evaluation at each step ensures quality and accuracy.

Evaluation Methods

Deterministic (Rule-based):

- Tool Correctness
- Field Coverage
- Type Compatibility
- Data Quality
- Null Analysis
- Duplicate Detection

DeepEval (LLM-based):

- Task Completion

Key Outputs

- **Agent 1:** Metadata & schema files
- **Agent 2:** Mapping plans & mapped CSVs
- **Agent 3:** final_mapped_dataset.csv
- **All Agents:** `output/workflow_sessions.db` captures the full conversation log for replay or auditing

Tools Used

Core Agents:

- load_and_describe_dataset - Data prep
- generate_mapped.csvs - Column mapping
- merged_mapped_csvs_to_target - Data integration
- core orchestration tools exposed via `tools/functions.py`

Evaluation:

- Agent-specific evaluation tools
- generate_summary_report - Final assessment

SQL-Alchemy

We use SQLAlchemy as the ORM layer to store all conversation logs and function calls generated by the agents. This ensures every agent run is traceable, reproducible, and auditable.

```python
# Create conversation ID for tracing
conversation_id = str(uuid.uuid4().hex[:16])
# Create a session instance with conversation ID.
session = SQLAlchemySession.from_url(
    conversation_id,
    url="sqlite+aiosqlite:///training_database.db",
    create_tables=True,
)
...
# Run the workflow using routing
with trace("Demand Forecasting Training", group_id=conversation_id):
    result = await Runner.run(agent, input=initial_message, session=session)
```

How It Works

- Creates a unique ID for the forecasting run.
- Opens a SQLAlchemy session to store logs in a SQLite database.
- Runs the agent and records everything that happens for traceability.

Why It Matters

Using SQLAlchemy allows us to:

- Maintain versioned records of all data mapping operations
- Debug forecasting errors by replaying previous sessions
- Track model performance across runs

Model Evaluation

Data Mapping Agent

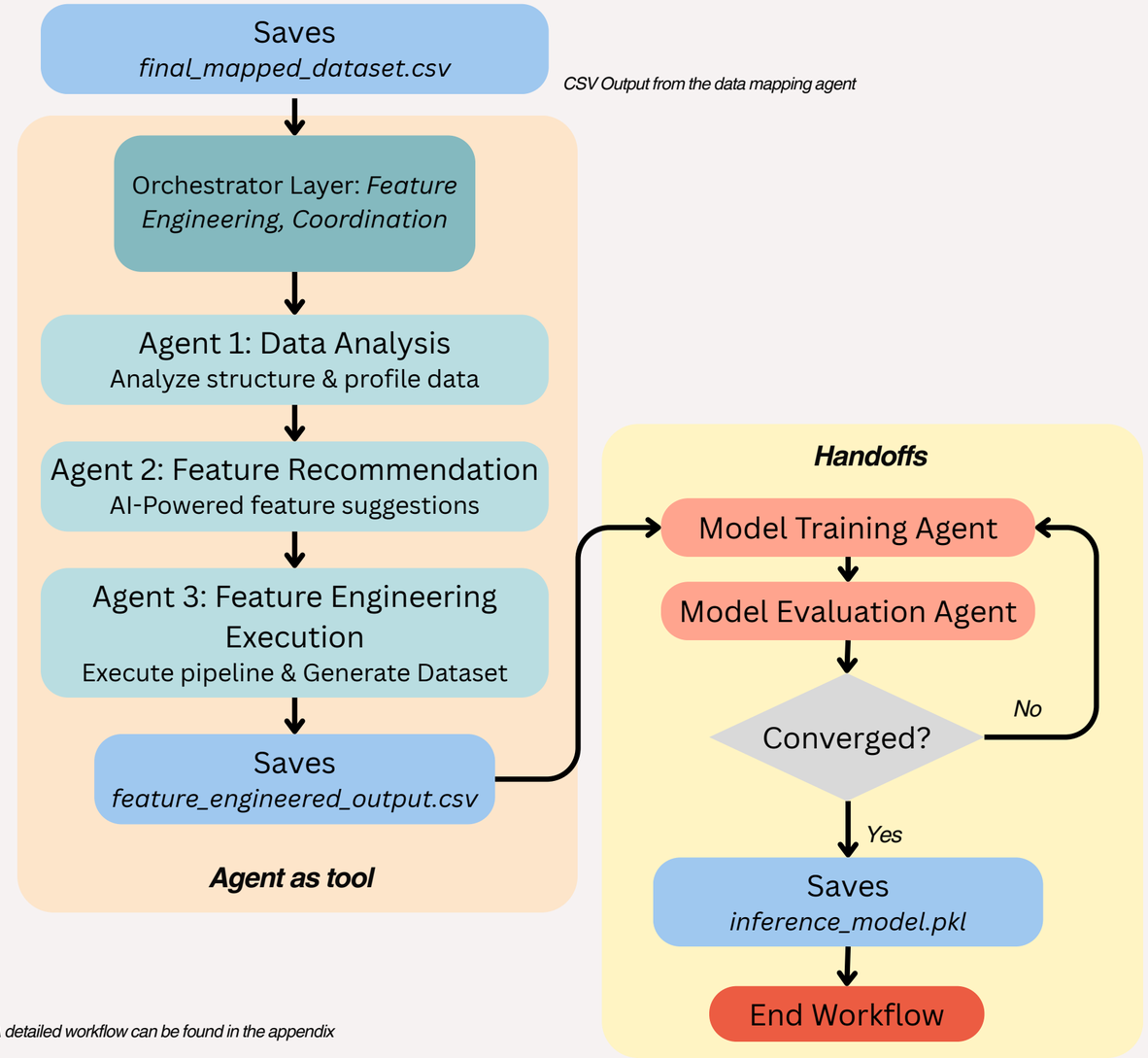
As a multi-agent workflow the evaluation is also divided in different agents for each part of the process, plus an additional agent that serves as a tool for the other agents to call. There's a set of tools that are shared among the evaluation agents and some specific for each task.

sample output

	Data Prep Eval Agent	Column Mapping Eval Agent	Data Integration Eval Agent
Task	Checks if the DataPrepAgent successfully loaded all CSV files and extracted complete metadata	Validates if the ColumnMappingAgent created semantically correct mappings with adequate field coverage	Verifies if the DataIntegrationAgent properly merged all datasets without data loss and validates the final CSV
Shared tools	**Task Completion Metric** (threshold: 0.6, LLM-based) • Did the agent successfully complete its assigned task? • Uses DeepEval to analyze if the agent's output fulfills the input requirements.	**Task Completion Metric** (threshold: 0.6, LLM-based) • Did the agent successfully complete its assigned task? • Uses DeepEval to analyze if the agent's output fulfills the input requirements.	**Summary report agent:** A meta-analysis tool that uses GPT-4 to interpret raw evaluation metrics and generate human-readable, actionable reports. Produces: Overall Assessment , metric analysis, success factors, failure analysis and recommendations.
Individual tools	**Answer Relevancy Metric** (threshold: 0.6, LLM-based) Is the metadata output relevant to the data profiling request? Ensures the agent stayed on-topic.	**Field Coverage Metric** (threshold: 0.9, deterministic) Percentage of target schema fields that were mapped **Type Compatibility Metric** (threshold: 1.0, deterministic) Are mapped column types compatible with target schema types? **Semantic Similarity Metric** (threshold: 0.5, token-based) How semantically similar are source column names to target column names? (e.g., "prod_id" → "product_id" = high)	**Row Preservation** (threshold: 0.95, deterministic) Were rows preserved during merge? **Field Coverage:** All required schema fields present in final CSV? **Null Analysis**: Any fields with >50% null values?

Model Methodology

Demand Forecasting Agent



✱ *A detailed workflow can be found in the appendix*

Overview

Automates end-to-end feature engineering and model training for demand forecasting.

- **Feature engineering sub-agents:**

Agent 1: Analyzes data structure and profiles data types
Agent 2: Generates AI-powered feature engineering recommendations
Agent 3: Executes complete feature engineering pipeline

- **Iterative model training loop** refines models until convergence, producing inference-ready models.

Training Methods

Model Training Agent:

- Creates time-series splits and trains models iteratively, incorporating feedbacks

Model Evaluation Agent:

- Assesses performance metrics, feature importance, and convergence

Key Outputs

Feature Engineering Agent:
feature_engineered_output.csv
Evaluation Agent:
inference_model.pkl (final model)
test_predictions.csv
feature_importance.png

Tools Used

Core Agents:
analyze_data_structure - Data profiling
feature_recommendation - AI feature suggestions
process_feature_engineering_pipeline - Execute pipeline

Model Training:
create_model_configs, load_and_preprocess_data
train_models, apply_feature_engineering
apply_hyperparameter_tuning, train_ensemble_models

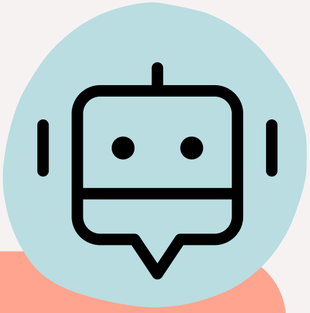
Evaluation:
evaluate_all_models - Performance assessment
check_convergence - Iteration control
save_best_model_for_inference - Final export
categorize_feedback - Improvement guidance

Model Evaluation

Demand Forecasting Agent

First Iteration

1. **Evaluate three models' performance** (XGBOOST, CATBOOST, LIGHTGBM) on validation set.
2. **Pick the best performing** one based on RMSE.
3. **Agent generate feedback** in one of the following areas:
    a. **Feature Engineering:** provide what features to add or remove or transform.
    b. **Hyperparameter** tuning: provide what hyperparameters to tune and the range of values to try.
    c. **Using Ensemble Modeling**: provide the base models to use and the type of ensemble to use.



Agent Feedback Example

"To improve the LightGBM model's performance, consider enhancing feature engineering by adding lag features (e.g., units_sold_lag_90, units_sold_lag_120), rolling window features beyond 30 days, creating more specific time-based features, and adding interaction terms between categorical features."

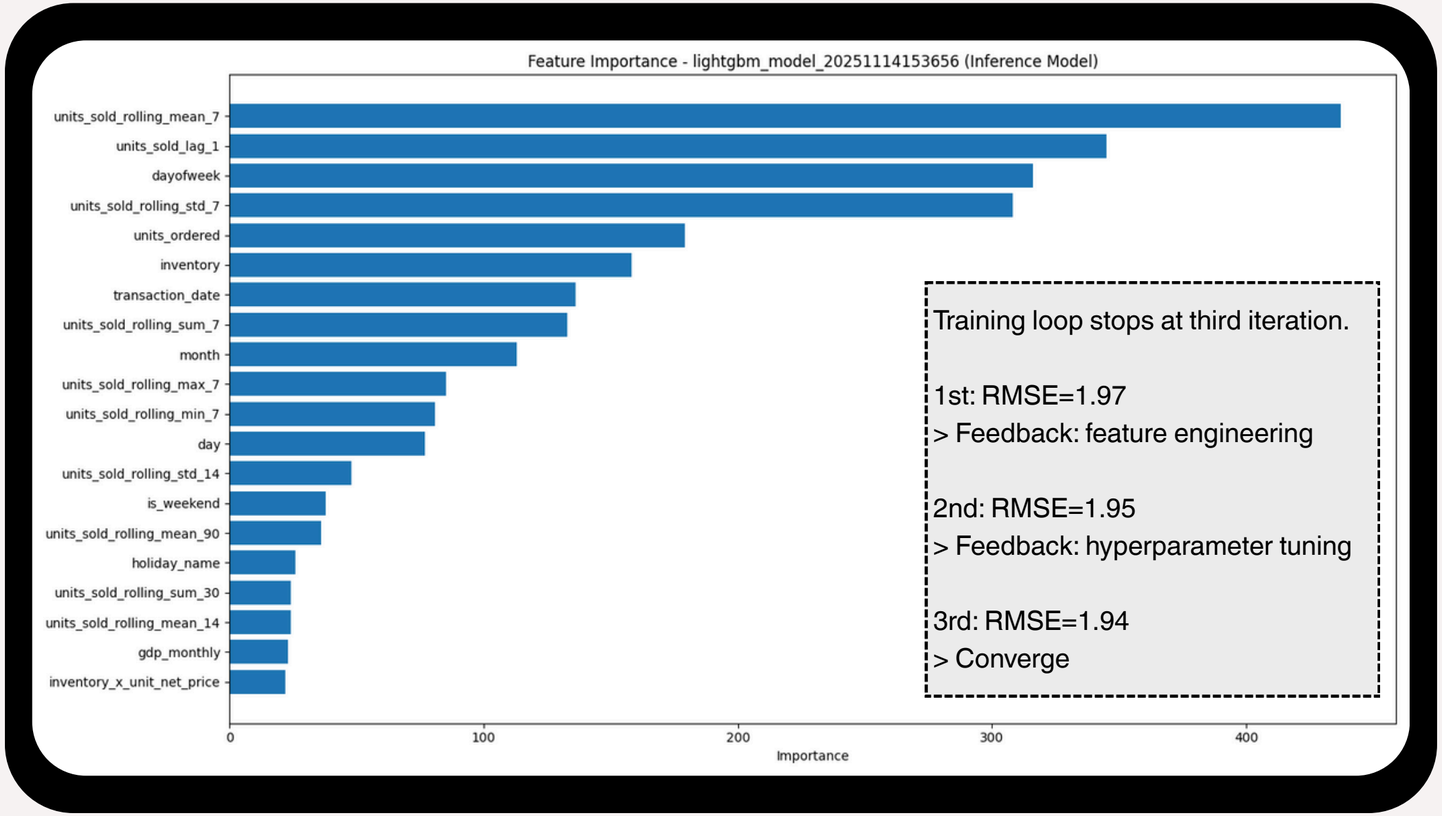
Subsequent Iteration

1. Evaluation focuses on the best performing model from the first iteration.
2. Check if the performance converges (improvement < 1%), comparing to the previous iteration.
3. If converges, retrain the model using training + validation set and save the model for inference.
4. If not, generate new feedback and continues the training loop.

Results

Feature Importance

This model uses **LightGBM**, a gradient-boosted tree algorithm that ranks features based on how much they reduce prediction error.



Top Predictors

- 7-Day Rolling Mean
- Log-transformed units sold
- Day of week

Model Performance

- RMSE (Validation): 1.94
- MAE (Validation): 1.38
- R^2 (Validation): 0.89

Additional Explorations

Google ADK

We explore Google ADK framework given some initial advantages we found like:

- **Simplified architecture:** No handoff filter is needed and there's and automatic state management that allows us to evaluate directly each step without manual history passing or complex filter functions.
- **Automatic state injection:** The syntax eliminate manual `HandoffInputData` objects and `to_input_list()` calls, making data flow between agents seamless and reducing error-prone state transformations by automatically capturing and injecting agent outputs.
- **Built in Orchestration:** The SequentialAgent provides deterministic execution with framework-managed error handling, retry logic, and observability features, eliminating the need for custom async/await coordination

Some downsides to this approach were:

- **Limited documentation:** Fewer tutorials make troubleshooting difficult, increasing development time when encountering uncommon issues
- **Reduced flexibility and framework lock in:** The sequential agent architecture makes it difficult to implement complex conditional logic. This proves to be specially challenging when integrating the evaluation and feedback loop meant for the system, since under this framework it was hard to code an optional feedback loop
- **Additional Abstraction Layer & Dependencies:** Requires LiteLLM wrapper to access OpenAI models and adds extra dependencies , creating additional abstraction overhead that can complicate debugging, potentially impact performance when integrating with forecasting agent system

LLM Match Artifact

The LLMatch approach represented an earlier iteration of the schema mapping system that leveraged OpenAI's GPT models to provide semantic understanding of column relationships through a three-stage pipeline inspired by the LLMatch methodology. The process is breakdown in three different parts

Table selection

This stage analyzes a source table's structure and semantically compares it against multiple candidate target tables to determine which ones are most relevant for mapping.

Rollup analysis

This stage identifies multiple source columns that should be combined or consolidated into a single target field by analyzing semantic relationships across the entire schema. The LLM detects patterns like separate "first_name" and "last_name" columns that should merge into "customer_name"

Drilldown

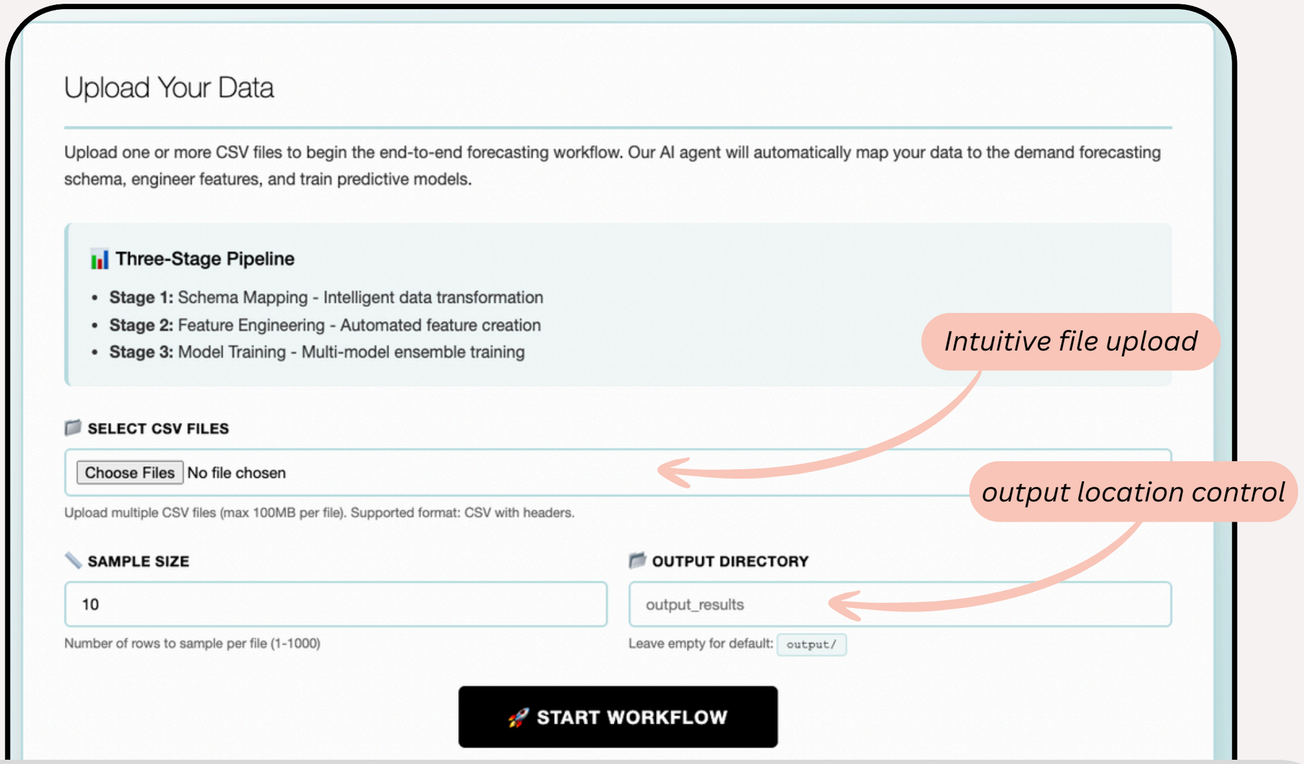
Once grouped columns are matched to a target field, this stage drills down to determine how each individual column within the group contributes to populating that target field.

Column matching

For each source column, the system passes the column name, data type, and description to the LLM alongside the complete target schema, asking it to identify potential mapping candidates through semantic analysis.

User Interface

Our **prototype** shows that multi-agent AI can be delivered through a simple, user-friendly interface, making advanced workflows accessible and improving efficiency.



Key Business Benefits

Accessibility

- No coding required
- One-click workflow execution

Cost Efficiency

- Self-service workflows reduce IT dependency
- Fewer errors and less rework

Operational Efficiency

- Faster response to business needs
- Consistent, repeatable outputs
- Analysts run workflows independently

How it Works

File selection → Schema mapping → Feature engineering → Trained models

- Automatic file handling between pipeline stages
- Real-time progress tracking and status updates

Future Enhancements

Data Mapping

- Integrate UI with additional data sources and formats
- Add adaptive routing to re-plan, skip, or parallelize steps
- Implement deterministic validators before merging
- Gate workflow completion on validation success
- Optimize context sharing with lightweight agent summaries
- Add rules to backfill or flag high-null columns
- Integrate duplicate detection and key enforcement before merge

Forecasting Agent

- Automate the full MLOps cycle, including drift detection, scheduled retraining, and model versioning.
- Connect data engineering to cloud.
- Expand from batch predictions to real-time forecasting with streaming data pipelines.
- Expand the capacity to incorporate more areas of feedback.

Conclusion

Results

- Successfully automated data preprocessing pipeline
- Multi-agent forecasting system with optimal model selection
- Demonstrated end-to-end integration
- Viable prototype with working UI and demos

Business Impact

- AI/ML automation optimized resource utilization
- Faster time-to-insight for forecasting
- Scalable, adaptable system architecture
- One-click user interface for easy forecasting

Threats to Validity

- Limited data sources and row samples in prototype (10 sources)
- Potential overfitting to test schemas
- LLM hallucination risks in mapping
- Generalization to unseen data formats
- Scalability to enterprise-scale data

Lessons Learned

What worked well:

- Agent-based architecture provided flexibility
- LLM integration for semantic understanding
- Evaluation loops improved quality

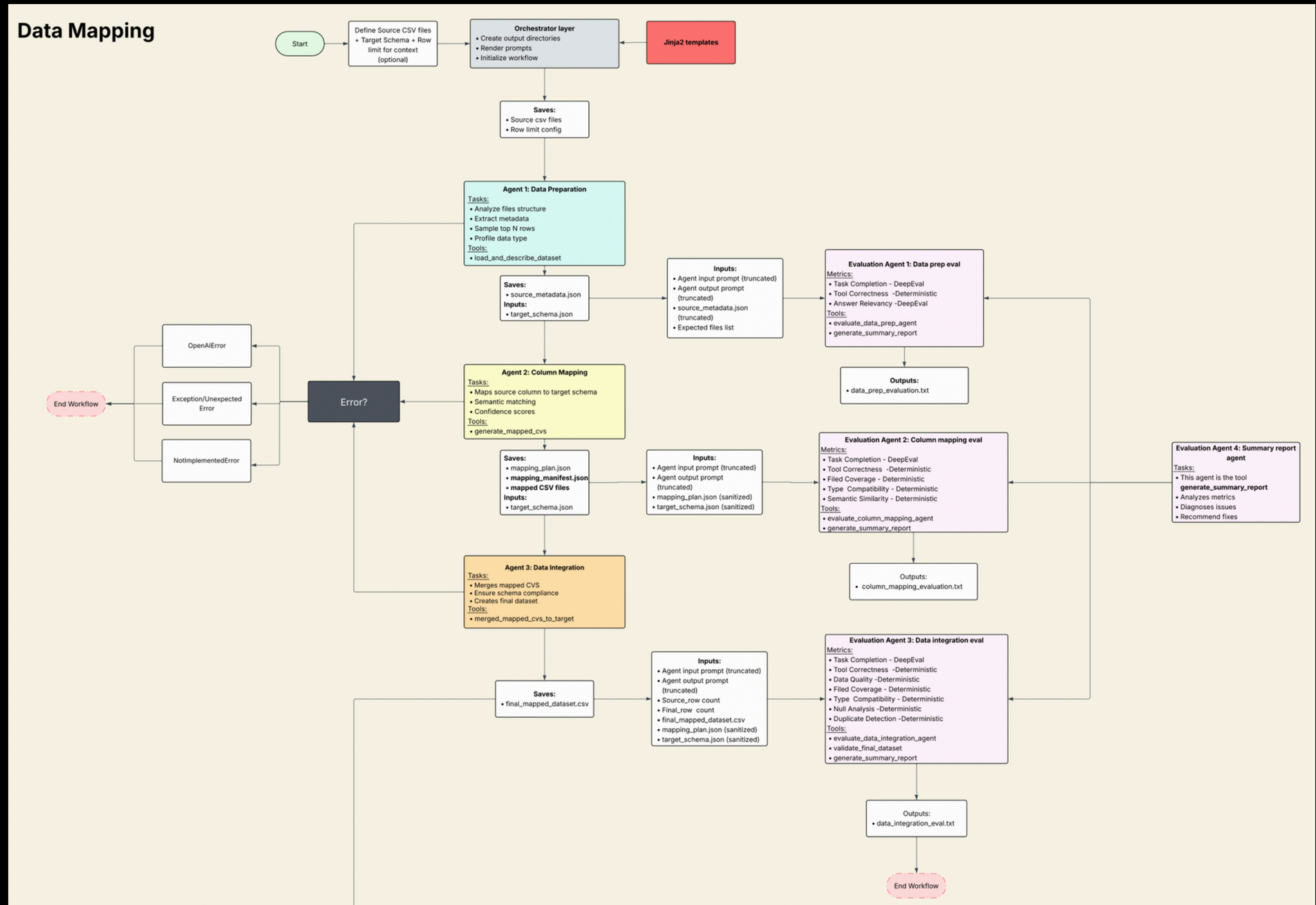
What was challenging:

- Agent coordination complexity
- Balancing automation with accuracy
- Handling edge cases in diverse data

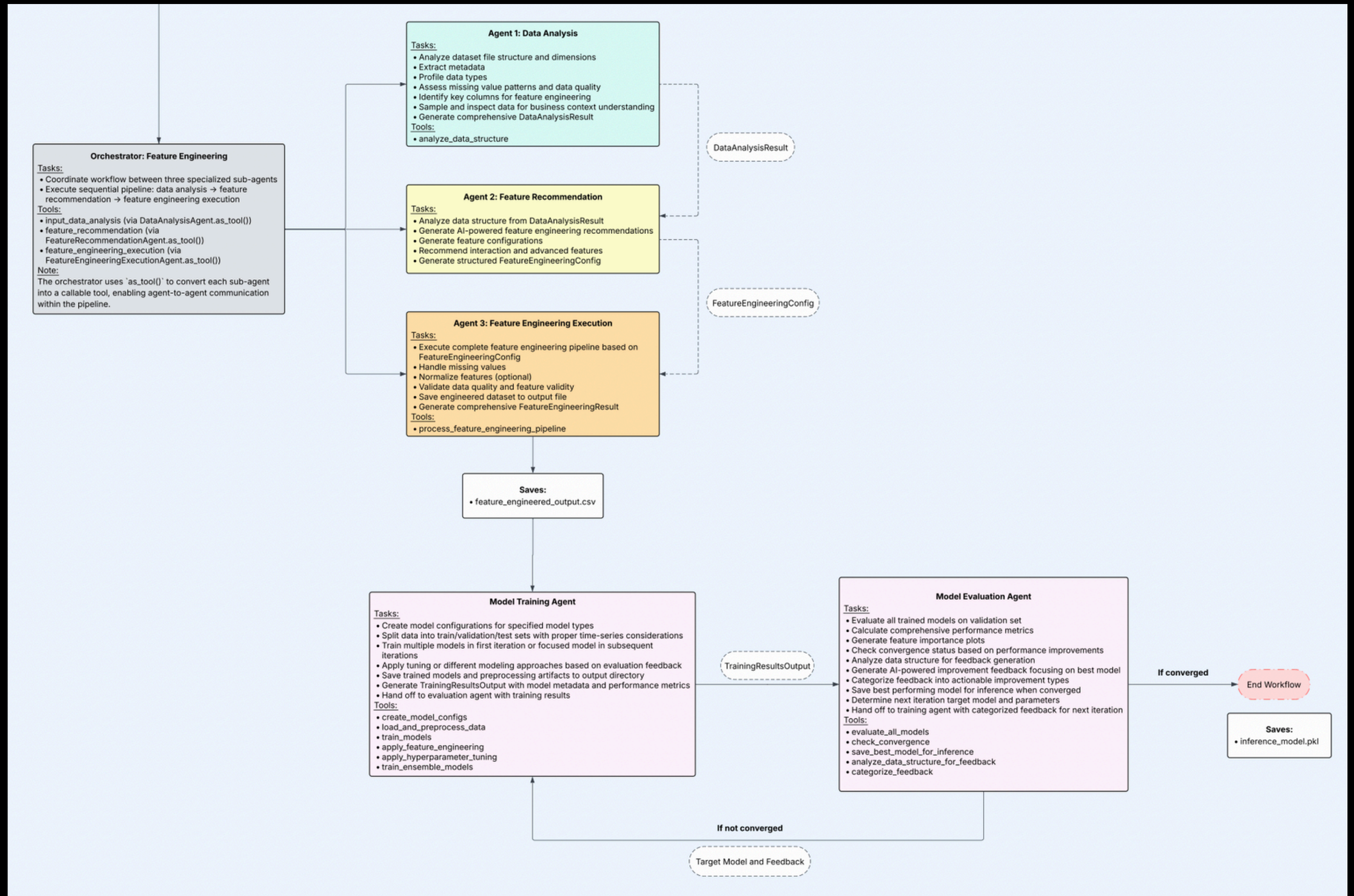
Appendix

Thank you!

A. Complete Data Mapping Workflow



B. Complete Demand Forecasting Workflow



C. Evaluation Strategy Breakdown

Component	What it does	Type	Comment
LLMTestCase	Stores Test Data	Built-in	Core component
Assert_test	Runs Evaluation	Built-in	Core component
BaseMetric	Template for custom metrics	Framework (build own eval logic)	
TaskCompletionMetric	Checks task completion	Built-in (uses LLM to judge)	Schema Mapping Agent, Demand Forecasting Agent
ToolCorrectnessMetric	Checks tool usage	Built-in (uses LLM to judge)	Schema Mapping Agent, Demand Forecasting Agent
GEval	Flexible LLM judge	Built-in (criteria customized, (uses LLM to judge)	Demand Forecasting Agent
AnswerRelevancyMetric	Checks if output addresses input	Built-in (uses LLM to judge)	Schema Mapping Agent, Demand Forecasting Agent (occasional use)
HallucinationMetric	Detects if agent makes up information not in context	Built-in (uses LLM to judge)	Demand Forecasting Agent
MAPE	Forecast accuracy %	Custom (with BaseMetric)	Demand Forecasting Agent
FieldCoverage	Field Mapping %	Custom (with BaseMetric)	Schema Mapping Agent
TypeCompatibility	Type checking	Custom (with BaseMetric)	Schema Mapping Agent
DirectionalAccuracy	Trend prediction	Custom (with BaseMetric)	Demand Forecasting Agent
SemanticSimilarityMetric	Checks if field mappings make semantic sense	Custom (with BaseMetric, or LLM)	Schema Mapping Agent
ConfidenceCalibrationMetric	Validates prediction interval accuracy	Custom (with BaseMetric)	Demand Forecasting Agent